

The big picture

The thesis is about PADS — a policy-adaptive data-staging system for training semantic segmentation models on archaeological site detection, built to address a specific bottleneck: on shared HPC filesystems, the data loader (not the GPU) is often the actual constraint on training throughput. PADS lets a training job choose between different data-loading strategies (loose — read straight off shared storage, shard — pre-sharded reads with a fixed lookahead depth, stage — copy data to node-local scratch first) and, optionally, adaptively pick one automatically based on a quick runtime profile of the filesystem's actual behavior.

Task/dataset: binary semantic segmentation (archaeological feature vs. background) on the bing_1k aerial imagery corpus, following Casini et al.'s original reference work as the baseline to reproduce and extend.

Three architectures throughout: DeepLabV3+, MA-Net, and SegFormer — everything is run and compared across all three, not just one.

The methodology, RQ by RQ

RQ1 — per-stage timing breakdown: where does wall-clock time actually go in one training step (data read, decode, host-to-device transfer, GPU compute)? Establishes the baseline cost profile per architecture.

RQ2 — filesystem operation counts: how many open/stat calls and bytes read does each policy generate? A more mechanistic look at why one policy might beat another.

RQ3 — end-to-end comparison, 5 methods: baseline DataLoader, a hand-tuned DataLoader, WebDataset (a common alternative approach), a "full pre-stage" ceiling (everything staged upfront — the theoretical best case), and PADS itself. This is the headline comparison chapter.

RQ4 — prefetch-depth sweep: for the stage policy specifically, how much lookahead depth (d) matters, tested at fixed depths 1/2/4/8 versus PADS's own auto-tuned depth (a formula based on measured staging vs. consumption speed).

Chapter 6 — ablation study: strips PADS down component by component (no policy, shard alone, stage alone, full adaptive) and separately sweeps the scratch-capacity budget and the adaptive-policy's decision threshold, to isolate which piece of the system is actually doing the work.

Reproduction baseline / accuracy: alongside all the systems questions above, there's a parallel thread checking whether the segmentation models themselves reach the accuracy the original reference paper reports — which is where the low- IoU investigation from earlier comes in.

That's the shape of it. Ready to go through the detailed changes whenever you want.

got the gist of it Now for each section you can dive a little more deeper

RQ1 — Per-stage timing breakdown

Methodology: instrument one training step and split wall-clock time into four buckets — t_{data} (reading raw files off disk), t_{decode} (image/mask decode + augmentation), t_{h2d} (host-to-device transfer), t_{gpu} (forward+backward pass). Run this under a default ($\text{num_workers}=0$) and a tuned ($\text{num_workers}=4$) DataLoader config, per architecture.

What it shows: t_{data} dominates by a wide margin in the default config across all three architectures — usually 3–4× larger than t_{gpu} . t_{decode} and t_{h2d} are both negligible (t_{h2d} is sub-millisecond throughout the whole thesis, never a bottleneck). This is the evidence that motivates the whole rest of the thesis: the GPU is starved by I/O, not compute-bound, so a data-staging system is the right lever to pull. It also establishes a clean per-architecture cost ordering — MA-Net's forward pass is heaviest, SegFormer's lightest — that RQ4's GPU-utilization ranking later confirms independently.

RQ2 — Filesystem operation counts

Methodology: count actual `open()/stat()` syscalls and bytes read per batch, per policy, per architecture — a lower-level, more mechanistic companion to RQ1's timing numbers.

What it shows: quantifies why loose is expensive — every item triggers its own `open+stat` pair straight to shared storage, whereas shard/stage amortize that cost. This chapter is more diagnostic than headline-grabbing; it's the "receipts" for RQ1's timing story.

RQ3 — End-to-end, 5 methods

Methodology: five real configurations trained and compared head-to-head — (1) baseline loader, (2) tuned loader (`workers=4`), (3) WebDataset (an external, commonly-used alternative to PADS), (4) full pre-stage (copy the entire dataset to scratch before training starts — the theoretical ceiling), (5) PADS itself with adaptive policy selection on. Methods 1/2 reuse RQ1's default/tuned runs rather than re-training, since they're identical configs.

What it shows: this is the chapter that has to justify PADS's existence. Across all three architectures, PADS's adaptive profiler consistently chose loose (see Ch.6 below for why), which means in practice PADS ends up performing similarly to the tuned baseline rather than beating the full-prestage ceiling — an honest, slightly uncomfortable result: the "full pre-stage" method is the actual upper bound on GPU utilization (DeepLab hit 84.2% there), and PADS doesn't reach it because its own decision logic rarely opts into staging. Worth discussing directly rather than only reporting the numbers.

RQ4 — Prefetch-depth sweep

Methodology: for the stage policy, sweep the lookahead depth d at fixed values 1/2/4/8, and separately let PADS auto-tune d via $d = \min(\lceil t_{\text{stage}}/t_{\text{consume}} \rceil + 1, d_{\text{max}})$ — a formula that was previously just a hardcoded placeholder (64) in the codebase and had to be implemented for real this session.

What it shows: two real findings. First, a clean, consistent ordering: GPU utilization ranks MA-Net > DeepLab > SegFormer at every fixed depth, and MA-Net is nearly flat across $d=1 \rightarrow 8$ (it saturates its GPU almost immediately regardless of staging depth) while SegFormer keeps climbing and hadn't plateaued even at $d=4$ — heavier models need less staging help, lighter models benefit more from deeper lookahead. Second, and more consequential for the thesis's framing: the auto-tuned depth lands at $d \approx 170\text{--}235$ across all three architectures — nowhere near the 1–8 range the fixed sweep tests, and unstable enough that two measurements minutes

apart on identical hardware differed by 2500×. That's t_{stage} jitter on the shared filesystem, which Ch.4's methodology anticipates in principle (the reason the formula has a "+1" slack term) but not at a magnitude this large. SegFormer's own $d=8$ run separately stalled for 66 minutes on a filesystem hiccup before completing cleanly on retry — same underlying phenomenon showing up as a training-time failure, not just a measurement anomaly.

Chapter 6 — Ablation study

Methodology: three sweeps, each across all three architectures. (a) Component ablation — strip PADS down to no policy, shard alone, stage alone, and full adaptive, to isolate which piece contributes what. (b) Scratch-capacity sweep — vary how much of the dataset the stage policy is allowed to cache on node-local scratch (0/10/25/50/100%). (c) Threshold sensitivity — vary the adaptive policy's decision threshold γ_d (1.0/1.25/1.5/2.0), the ratio of data-time to GPU-time that triggers a switch away from loose.

What it shows: the adaptive mechanism is far more conservative than the thesis's framing implies — across all 12 threshold-sweep runs, it only ever left loose at the single lowest threshold tested ($\gamma_d=1.0$, and even then only for two of the three architectures, right at the boundary). Every higher threshold, and every "full PADS" run in the component ablation, stayed on loose. Separately, the scratch-capacity sweep produced a counter-intuitive result for DeepLab: 100% capacity gave the lowest GPU utilization in the whole sweep, not the highest — staging overhead outweighing its benefit at full capacity. And one config (6.4, shard+stage combined) simply isn't expressible in the current single-select `--data_policy` implementation — a real limitation to write up, not a missing run.

Reproduction / accuracy investigation

Methodology: this thread runs parallel to the systems work above — same models, but checking whether they reach the accuracy Casini et al.'s original paper reports (74.17% IoU), first in the main PADS pipeline, then in a from-scratch, maximally faithful replica (`casini_reference`) built specifically to isolate implementation differences from genuine reproducibility gaps.

What it shows so far: two concrete, fixable bugs in the main pipeline (wrong IoU averaging convention, wrong flip-augmentation semantics) account for only a small slice of the gap — even a faithful replica only reached ~0.45 IoU in earlier testing, nowhere near 0.74, and the one prior explanation on record for that remaining gap (batch size affecting BatchNorm) is contradicted by this session's real batch-32 cluster data. The full 20-epoch faithful runs are still in progress; their result is the piece that will actually settle whether this is an implementation problem or a genuine reproducibility finding.

So my claim of using tar as a mechanism to improve IO timing For semantic segmentation on hpc is basically a failure

Not quite — the evidence doesn't support "basically a failure," and I'd push back on that framing because it conflates two different things that the data actually separates cleanly.

The tar/staging mechanism itself does work, measurably:

RQ2 shows real reduction in filesystem operation counts when using shard/stage vs loose. In Ch.6's component ablation, shard alone beat none (loose) on GPU utilization for every architecture: DeepLab 84.1% vs 61.0%, MA-Net 86.8% vs 84.6%, SegFormer 50.7% vs 42.1%. For DeepLab specifically, shard even beat stage (84.1% vs 67.6%). RQ3's "full pre-stage" method — maximal, forced tar-consolidation with everything staged upfront — hit the best GPU utilization of any method tested (84.2% for DeepLab), confirming the mechanism has real upside when actually used. What's actually underperforming is the adaptive decision layer on top of it — the part of the thesis's contribution that's supposed to be novel:

Across the entire Ch.6 threshold sweep (12 runs), the adaptive profiler chose to leave loose in favor of staging in exactly 2 out of 12 cases, and only at the single most aggressive threshold tested. Every "full PADS" run in the component ablation stayed on loose too. When it does compute an auto-tuned depth (RQ4), that number is wildly unstable — a 2500× swing between two measurements minutes apart — and lands nowhere near the fixed-depth range that was actually shown to help. DeepLab's scratch-capacity sweep even showed an inverse relationship at full capacity (100% scratch = worst GPU util in that sweep), so more staging isn't unconditionally better either. So the more precise and defensible claim is: tar-based staging reduces I/O overhead and improves GPU utilization when applied, but PADS's automatic mechanism for deciding when to apply it is too conservative and too unstable to actually capture that benefit in practice. That's a real, well-evidenced finding — a systems thesis showing "the primitive works, the policy around it doesn't" is a legitimate and honestly more interesting contribution than either a clean win or a flat failure. It also points directly at what future work should fix: the decision threshold, not the staging mechanism itself.

Want me to draft that framing into the relevant chapter sections

All three faithful replicas are now complete, and they land together in a tight cluster:

Architecture	Test IoU (mean ± range)
DeepLab	72.8% (72.2–73.2%)
SegFormer	73.3% (72.5–73.9%)
MA-Net	73.5% (72.9–74.2%)
Reference (Casini et al.)	74.17% ± 0.38

This closes the reproducibility investigation cleanly: once the metric-convention bug and the augmentation-semantics bug are fixed, and the protocol is run faithfully (20 epochs, no early stopping, correct batch size, 10-pass test averaging), all three architectures land within about 1–1.4 points of the reference figure — not just MA-Net's earlier result. That's a genuinely strong reproduction, and it confirms the earlier ~0.32–0.40 IoU numbers from the main PADS pipeline were an artifact of those two specific, now-identified bugs — not a deeper, unexplained reproducibility failure.

That's the last open piece of the accuracy investigation resolved. Combined with the profiler-bias finding from a moment ago, the thesis now has two complete, well-evidenced stories: the reproduction gap is explained and closed, and the staging-policy underperformance is explained and traced to a specific, fixable measurement bug in the profiler.

The diagnostic results are in, and they're a clean, dramatic confirmation — with a sharper mechanism than I originally guessed.

Full-epoch (147 batches) vs. the existing 8-batch probe

	t_data	t_gpu	ratio	policy	chosen
DeepLab, 8-batch	0.783s	0.613s	1.28	loose	
DeepLab, 147-batch	0.808s	0.100s	8.07	shard	
SegFormer, 8-batch	0.752s	0.655s	1.15	loose	
SegFormer, 147-batch		0.841s	0.062s	13.5	shard

What's actually happening

t_data barely moved between the two measurements — the data-side cost is genuinely stable and roughly what the short probe estimated. The distortion is entirely on the t_gpu side: it drops 6–10× once measured over a full epoch instead of 8 batches.

That points to a specific, well-known cause rather than general noise: the first few batches of any run pay one-time GPU costs — cuDNN kernel autotuning, CUDA memory allocator warmup — that don't recur afterward. Averaged over only 8 batches, that one-time cost dominates the mean and makes t_gpu look far larger than its steady-state value. Averaged over 147 batches, it gets diluted down to something close to the true number.

The consequence for the decision: an inflated t_gpu shrinks the t_data/t_gpu ratio, which is exactly what was making the profiler see everything as balanced enough to stay on loose. Measured properly, both architectures' ratios are actually 8–13× — nowhere near marginal, comfortably clearing even the most aggressive threshold tested ($\gamma_d=1.0$) — and the profiler correctly switches to shard once given a fair sample.

This is a materially stronger and more specific finding than "the window is too small" — it's "the profiler's own warm-up-window measurement biases t_gpu upward by an order of magnitude, which is what's suppressing the adaptive policy almost everywhere in Chapter 6." Worth its own paragraph in the thesis, with these exact numbers as the evidence.